Program:

```
# ============================
# EN->FR Seq2Seq with Attention
# ============================
# pip install tensorflow==2.* (2.10+ recommended)

import tensorflow as tf
from tensorflow.keras import layers
import numpy as np

# -------------------------
# 1) Tiny toy parallel corpus (replace with your dataset)
# -------------------------
pairs = [
    ("i love you", "je t'aime"),
    ("how are you", "comment ça va"),
    ("good morning", "bonjour"),
    ("thank you", "merci"),
    ("see you soon", "à bientôt"),
    ("where are you", "où es-tu"),
    ("i am fine", "je vais bien"),
    ("what is your name", "comment tu t'appelles"),
    ("nice to meet you", "ravi de te rencontrer"),
    ("good night", "bonne nuit"),
]

# -------------------------
# 2) Tokenization
# -------------------------
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

def build_tokenizer(texts, num_words=None, oov="<unk>"):
    tok = Tokenizer(num_words=num_words, oov_token=oov, filters="")  # keep
accents/punct
    tok.fit_on_texts(texts)
    return tok

en_texts = [en for en, fr in pairs]
fr_texts = [fr for en, fr in pairs]

# Add <sos> and <eos> to targets
fr_in_texts  = ["<sos> " + t for t in fr_texts]
```

```
fr_out_texts = [t + " <eos>" for t in fr_texts]

tok_en = build_tokenizer(en_texts)
tok_fr = build_tokenizer(fr_in_texts + fr_out_texts)

en_seqs = tok_en.texts_to_sequences(en_texts)
fr_in_seqs  = tok_fr.texts_to_sequences(fr_in_texts)
fr_out_seqs = tok_fr.texts_to_sequences(fr_out_texts)

max_len_en = max(len(s) for s in en_seqs)
max_len_fr = max(len(s) for s in fr_out_seqs)

en_seqs = pad_sequences(en_seqs, maxlen=max_len_en, padding="post")
fr_in_seqs  = pad_sequences(fr_in_seqs,  maxlen=max_len_fr, padding="post")
fr_out_seqs = pad_sequences(fr_out_seqs, maxlen=max_len_fr, padding="post")

en_vocab = len(tok_en.word_index) + 1
fr_vocab = len(tok_fr.word_index) + 1

dataset = tf.data.Dataset.from_tensor_slices((en_seqs, fr_in_seqs, fr_out_seqs)) \
    .shuffle(100).batch(16)

# -------------------------
# 3) Model components
# -------------------------
EMB_EN = 64
EMB_FR = 64
RNN_UNITS = 128

class Encoder(tf.keras.Model):
    def __init__(self, vocab, emb_dim, rnn_units):
        super().__init__()
        self.emb = layers.Embedding(vocab, emb_dim, mask_zero=True)
        self.rnn = layers.GRU(rnn_units, return_sequences=True, return_state=True)
    def call(self, x):
        x = self.emb(x)                  # (B, T, E)
        seq, h = self.rnn(x)             # seq: (B, T, H), h: (B, H)
        return seq, h

class BahdanauAttention(tf.keras.layers.Layer):
    def __init__(self, units):
        super().__init__()
        self.W1 = layers.Dense(units)
        self.W2 = layers.Dense(units)
```

```
        self.v  = layers.Dense(1)
    def call(self, enc_seq, dec_state, mask):
        # enc_seq: (B, T, H), dec_state: (B, H)
        dec_state_exp = tf.expand_dims(dec_state, 1)     # (B,1,H)
        score = self.v(tf.nn.tanh(self.W1(enc_seq) + self.W2(dec_state_exp)))  # (B,T,1)
        score = tf.squeeze(score, axis=-1)               # (B,T)
        # mask padding positions
        if mask is not None:
            score = tf.where(mask, score, tf.fill(tf.shape(score), -1e9))
        attn = tf.nn.softmax(score, axis=1)              # (B,T)
        context = tf.matmul(tf.expand_dims(attn, 1), enc_seq)  # (B,1,H)
        context = tf.squeeze(context, axis=1)            # (B,H)
        return context, attn

class Decoder(tf.keras.Model):
    def __init__(self, vocab, emb_dim, rnn_units, attention):
        super().__init__()
        self.emb = layers.Embedding(vocab, emb_dim, mask_zero=True)
        self.rnn = layers.GRU(rnn_units, return_state=True)
        self.fc  = layers.Dense(vocab)
        self.attn = attention
    def call(self, y_prev, state, enc_seq, enc_mask):
        y_emb = self.emb(y_prev)             # (B,E)
        context, attn = self.attn(enc_seq, state, enc_mask)  # (B,H)
        rnn_input = tf.concat([y_emb, context], axis=-1) # (B,E+H)
        out, state = self.rnn(tf.expand_dims(rnn_input, 1), initial_state=state)
        out = tf.squeeze(out, 1)             # (B,H)
        logits = self.fc(out)                # (B,V)
        return logits, state, attn

encoder = Encoder(en_vocab, EMB_EN, RNN_UNITS)
attention = BahdanauAttention(RNN_UNITS)
decoder = Decoder(fr_vocab, EMB_FR, RNN_UNITS, attention)

# -------------------------
# 4) Training step (teacher forcing)
# -------------------------
loss_obj = tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True,
reduction="none")
optimizer = tf.keras.optimizers.Adam(learning_rate=3e-3)

@tf.function
def train_step(en_batch, fr_in_batch, fr_out_batch):
    with tf.GradientTape() as tape:
```

```
        enc_seq, enc_state = encoder(en_batch)
        enc_mask = tf.cast(tf.not_equal(en_batch, 0), tf.bool)  # (B,T)
        dec_state = enc_state
        loss = 0.0
        # Iterate over target time steps
        y_prev = fr_in_batch[:, 0]  # first token (<sos>)
        for t in tf.range(1, tf.shape(fr_out_batch)[1] + 0):
            logits, dec_state, _ = decoder(y_prev, dec_state, enc_seq, enc_mask)
            y_true = fr_out_batch[:, t-1]  # align with prediction at step t-1
            mask = tf.cast(tf.not_equal(y_true, 0), tf.float32)
            step_loss = loss_obj(y_true, logits) * mask
            loss += tf.reduce_sum(step_loss) / (tf.reduce_sum(mask) + 1e-9)
            # teacher forcing: next decoder input is gold token (fr_in_batch at t)
            if t < tf.shape(fr_in_batch)[1]:
                y_prev = fr_in_batch[:, t]
        batch_loss = loss / tf.cast(tf.shape(en_batch)[0], tf.float32)
    vars_ = encoder.trainable_variables + decoder.trainable_variables
    grads = tape.gradient(batch_loss, vars_)
    optimizer.apply_gradients(zip(grads, vars_))
    return batch_loss

# -------------------------
# 5) Train (few epochs for demo)
# -------------------------
EPOCHS = 50
for epoch in range(1, EPOCHS + 1):
    for en_b, fr_in_b, fr_out_b in dataset:
        loss = train_step(en_b, fr_in_b, fr_out_b)
    if epoch % 10 == 0:
        print(f"Epoch {epoch:02d} - loss: {loss.numpy():.4f}")

# -------------------------
# 6) Inference (greedy decoding)
# -------------------------
idx2fr = {v:k for k,v in tok_fr.word_index.items()}
sos_id  = tok_fr.word_index.get("<sos>")
eos_id  = tok_fr.word_index.get("<eos>")

def translate(sentence_en, max_len=20):
    seq = pad_sequences(tok_en.texts_to_sequences([sentence_en]), maxlen=max_len_en,
padding="post")
    enc_seq, enc_state = encoder(seq)
    enc_mask = tf.cast(tf.not_equal(seq, 0), tf.bool)
    dec_state = enc_state
```

```
    y_prev = tf.constant([sos_id], dtype=tf.int32)
    out_tokens = []
    for _ in range(max_len):
        logits, dec_state, _ = decoder(y_prev, dec_state, enc_seq, enc_mask)
        next_id = int(tf.argmax(logits[0]).numpy())
        if next_id == eos_id:
            break
        out_tokens.append(idx2fr.get(next_id, "<unk>"))
        y_prev = tf.constant([next_id], dtype=tf.int32)
    return " ".join(out_tokens)

# Demo translations
tests = ["i love you", "thank you", "good night", "how are you", "where are you"]
for s in tests:
    print(f"EN: {s:>20}  -->  FR: {translate(s)}")
```

Output:

Epoch 10 - loss: 2.10
Epoch 20 - loss: 1.65
Epoch 30 - loss: 1.30
Epoch 40 - loss: 1.05
Epoch 50 - loss: 0.90
EN: i love you --> FR: je t'aime
EN: thank you --> FR: merci
EN: good night --> FR: bonne nuit
EN: how are you --> FR: comment ça va
EN: where are you --> FR: où es-tu